Domain 3: Claude Code Configuration & Workflows — Practice-Based Self-Questioning

---

1. Add a date validation conditional to one function in one file

- Self-question: For a simple, well-scoped, single-file change, is direct execution the right choice rather than plan mode or extended thinking?
- Key distinction: direct execution (simple/low-risk) vs. plan mode (unnecessary overhead here) vs. extended thinking — what makes a task "not worth planning"?

2. Auth library v2→v3 migration across 45 files, breaking changes

- Self-question: Does a large, multi-decision migration across many files call for plan mode to map affected paths first — rather than fixing failures reactively, pasting the guide and direct-executing, or a slash command without exploration?
- Key distinction: plan mode (large scope, multiple decisions) vs. fix-as-tests-fail vs. direct execution with the migration guide vs. blind slash command — what makes scope + interacting changes demand upfront analysis?

3. How MCP prompts become accessible in Claude Code

- Self-question: Do MCP prompts appear as slash commands (`/mcp__server__prompt`) invoked explicitly — rather than @-mentionable resources, auto-prepended context, or tool-registry entries?
- Key distinction: slash commands vs. @-mentionable resources vs. auto-prepended system context vs. tool registry auto-invocation — which one is the real access mechanism?

4. Shared venue server (team) + personal experimental playlist server

- Self-question: Does the shared server go in `.mcp.json` (version-controlled) and the personal one in `~/.claude.json` (not shared)?
- Key distinction: `.mcp.json` (project/shared) vs. `~/.claude.json` (user/personal) — and watch for the reversed-order distractor.

5. Critical production bug in an unfamiliar module with a clear stack trace

- Self-question: Even with a clear stack trace, does working in an *unfamiliar* module justify plan mode to enumerate root causes systematically before fixing?
- Key distinction: plan mode (unfamiliar architecture) vs. direct execution (you'd be guessing at design) vs. hybrid direct-then-plan — what does "never worked with this module" imply for mode choice?

6. Complex algorithm with edge cases — iterative refinement

- Self-question: Is test-driven iteration (write the test suite first, then iterate on failures) the most effective workflow — rather than natural-language spec + manual feedback, or extended-thinking plan-then-implement?

- Key distinction: test-driven iteration (objective signals) vs. natural-language spec + descriptive feedback vs. extended-thinking full-solution vs. reference-implementation rewrite — which gives Claude unambiguous, repeatable signals?

7. 500-line root CLAUDE.md loading irrelevant rules per file type

- Self-question: Do path-scoped `.claude/rules/` files with YAML frontmatter load only relevant guidance per file type — better than subdirectory CLAUDE.md, @imports, or reorganizing the root file into sections?
- Key distinction: `.claude/rules/` path scoping vs. subdirectory CLAUDE.md vs. @imports vs. header sections in one big file — which minimizes tokens by loading conditionally on file path?

8. Interacting PDF formatting issues (columns, dates, page breaks)

- Self-question: When issues interact, should they be fixed one at a time with testing after each — rather than all at once, a fresh detailed prompt, or matching an example?
- Key distinction: incremental fixes + test-after-each vs. batching all fixes vs. starting fresh vs. example-matching — why do interacting/cascading problems require isolation?

9. Prettier rule in CLAUDE.md still yields ~15% inconsistency

- Self-question: Is a PostToolUse hook (Edit|Write matcher) running Prettier the fix — since CLAUDE.md is advisory and emphasis can't eliminate violations?
- Key distinction: PostToolUse hook (deterministic) vs. a skill vs. path-scoped rules vs. a Stop hook prompt-check — which achieves 100% compliance by removing model discretion?

10. Inconsistent ApiError convention across sessions — first diagnostic step

- Self-question: Is the most efficient first step running `/memory` to confirm the CLAUDE.md is actually loaded — before adding examples, path rules, or hunting for conflicts?
- Key distinction: `/memory` (verify loading first) vs. more examples vs. searching for conflicting instructions vs. path-scoped rules — why confirm the file loads before assuming it's the advisory-nature problem?

11. One-off task following patterns in three existing modules

- Self-question: For a one-off task where patterns don't need to be permanent project docs, do `@references` to the three files beat describing them, adding to CLAUDE.md, or asking Claude to explore?
- Key distinction: `@references` (temporary, precise) vs. natural-language description vs. CLAUDE.md (loads every session, unnecessary) vs. codebase exploration (slow) — which keeps the context one-off and concrete?

12. Accessing tools across three connected MCP servers in one request

- Self-question: Are tools from all configured MCP servers discovered at connection time and available simultaneously — rather than sequential routing, auto-selecting one server, or one-server-per-turn?
- Key distinction: all-tools-available-simultaneously vs. sequential prefix routing vs. auto-select one server vs. manual one-server-at-a-time — how does multi-server tool access actually work?

13. Rough caching idea, unsure of robust-implementation considerations

- Self-question: When requirements are uncertain, does asking Claude to interview you first (surfacing invalidation, consistency, failure modes) beat a TBD-marker spec, plan mode, or a minimal request?
- Key distinction: interview pattern vs. TBD-marker spec vs. plan mode (analyzes existing code, not your unknowns) vs. minimal request + iterate — which surfaces *unknown unknowns* before committing?

14. Three requirements mis-placed in CLAUDE.md (never-edit, prefer, always-format)

- Self-question: Should "never" → `permissions.deny`, "prefer" → CLAUDE.md, "always run after every edit" → PostToolUse hook — matching each requirement to its enforcement level?
- Key distinction: permissions.deny (hard block) vs. CLAUDE.md (advisory preference) vs. PostToolUse hook (deterministic automation) — and why putting all three in CLAUDE.md fails the "never" requirement.

15. Migration script mishandles null values — how to iterate

- Self-question: Does providing a concrete test case (input with nulls + expected output) beat describing the problem, "think harder," or manually editing?
- Key distinction: concrete test case vs. detailed description + regenerate vs. "think harder about edge cases" vs. manual edit — why does showing the exact expected behavior outperform describing it?

16. Real-time updates: WebSockets vs. SSE vs. polling

- Self-question: When multiple approaches with different trade-offs exist, does plan mode (explore, evaluate, present options) fit better than direct execution of any single choice?
- Key distinction: plan mode (multiple approaches, trade-off analysis) vs. direct-execute polling vs. direct-execute WebSockets vs. "let Claude pick and implement" — why does genuine architectural choice need analysis first?

17. Team-shared /migrate-component workflow that stays in sync

- Self-question: Does the skill belong in `.claude/skills/migrate-component/SKILL.md` at the project root, committed to version control — rather than user-level, settings.json, or root CLAUDE.md?
- Key distinction: project-level skill (shared, versioned, on-demand) vs. `~/.claude/skills/` (personal) vs. settings.json (not how skills work) vs. CLAUDE.md (loads every session) — which is shared *and* on-demand?

18. 8-item review checklist used only during PR reviews

- Self-question: Does a `/review` slash command (loaded only when invoked) fit an occasional workflow better than a subagent, CLAUDE.md, or default plan mode?
- Key distinction: slash command/skill (on-demand) vs. dedicated subagent vs. CLAUDE.md (always loaded, wasteful) vs. default plan mode — why keep an occasional checklist out of always-loaded context?